Datenstrukturen: Array, Liste, Stack, Queue, Baum

BEGRIFFE

Datenstruktur: legt mehrere Daten, auch verschiedener Typen, gemeinsam im Speicher ab (String, Struktur „Kunde“, Tabelle). int = Datentyp, Array von int = Datenstruktur.

ARRAY

- Fortlaufender **Index ab 0**, **feste Länge**, direkter Zugriff über a[i].
- + sehr schneller Zugriff. – Einfügen/Entfernen nur durch Verschieben/Umkopieren; voll → neues, größeres Array.

VERKETTETE LISTE

- Kein Index, nur der **Kopf** ist bekannt. Jedes Element: Daten + Zeiger **next** (Adresse des Nachfolgers), letztes → null.
- + Einfügen/Löschen nur Zeiger umhängen, Länge flexibel. – Lesen: vom Kopf aus durchlaufen.

X hinter A einfügen (Reihenfolge!):

1. X.next = A.next
2. A.next = X

B löschen: Vorgänger A suchen, A.next = B.next.

Doppelt verkettet: prev und next; beide Richtungen, Vorgänger beim Löschen nicht suchen; dafür mehr Speicher.

Sortierte Liste: vom Kopf laufen bis zum ersten größeren Element, davor einhängen. 10 → 19 → 36, 25 einfügen: 36 > 25 → stopp → 10 → 19 → 25 → 36.

STACK UND QUEUE

	Stack	Queue
Prinzip	**LIFO** (Last In, First Out)	**FIFO** (First In, First Out)
Rein	push(x): oben drauf	put(x)/enqueue: hinten an
Raus	pop(): oberstes nehmen	get()/dequeue: vorderstes nehmen
Lesen	top(): oberstes lesen, bleibt	–
Fehler	pop/top auf leerem Stack	get auf leerer Queue
Bau	Array + Index oder Liste	Ringpuffer im Array

Durchspielen: Zustand nach jeder Operation notieren, Ausgaben mitschreiben.

- Stack: push 4, 8, 2, pop, push 5, top, pop, pop → Ausgaben **2, 5, 5, 8**, Rest [4].
- Queue: put 4, 8, 2, get, put 5, get, get → Ausgaben **4, 8, 2**, Rest [5].

BAUM: BEGRIFFE

- **Wurzel:** Ursprung, oben, Ebene 0. Jeder andere Knoten hat genau einen Vorgänger.
- **Blatt:** ohne Nachfolger. **Innerer Knoten:** mindestens ein Nachfolger.
- **Ebene:** Kanten von der Wurzel bis zum Knoten. **Höhe:** höchste Ebene. **Pfadlänge:** Summe der Ebenen aller Knoten.

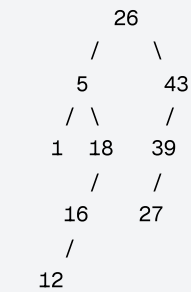
GEORDNETER BINÄRBAUM

Höchstens 2 Kinder; links nur kleinere, rechts nur größere Werte.

Einfügen/Suchen:

1. An der Wurzel beginnen.
2. Kleiner → links, sonst rechts.
3. Bis ein freier Platz kommt, dort als Blatt anhängen.

[26, 5, 18, 16, 1, 43, 39, 27, 12] in dieser Reihenfolge:



Höhe 4, Blätter 1, 12, 27. **Probe:** links – Knoten – rechts gelesen ergibt 1, 5, 12, 16, 18, 26, 27, 39, 43.

Löschen:

1. Blatt: entfernen.
2. Ein Kind: Kind rückt mit Unterbaum nach (18 löschen → 16 mit 12 rückt hoch).
3. Zwei Kinder: ersetzen durch **größten Wert links** (1× links, dann ganz rechts) oder **kleinsten Wert rechts** (1× rechts, dann ganz links); Ersatz an alter Stelle nach Fall 1/2 entfernen. Variante angeben.

- **voll:** jeder Knoten 0 oder 2 Kinder. **vollständig:** voll + alle Blätter auf der letzten Ebene.
- **balanciert:** Blätter auf letzter oder vorletzter Ebene; Tiefe ≈ $\log_2(n + 1)$, 1 000 Knoten ≈ 10 Ebenen.
- **degeneriert:** jeder Knoten höchstens 1 Kind = Liste (sortierte Eingabe 1, 2, 3, 4).

VERGLEICH

Struktur	Zugriff	+ / –
Array	O(1)	schnell / starr
Liste	O(n)	flexibel / langsame Suche
sortierte Liste	O(n)	sortiert / Einfügen teurer
Stack, Queue	nur oben/vorne	einfach / kein freier Zugriff
Binärbaum	balanciert O(log n)	schnell / kann degenerieren

TYPISCHE FEHLER

- a[10] bei 10 Elementen: Index endet bei 9.
- Beim Einfügen zuerst A.next umbiegen → Rest der Liste verloren.
- LIFO und FIFO vertauschen; top() entfernt nichts, pop() schon.
- Fehlerfall (leere Struktur) vergessen.
- Ebenen ab 1 zählen statt ab 0.
- Zahlen nicht in der gegebenen Reihenfolge einfügen.
- Bei zwei Kindern einfach das linke Kind hochziehen.
- Probe vergessen: von links nach rechts muss die Folge sortiert sein.